1 Overview

One of the major areas of computer vision is 3D reconstruction. Given several 2D images of an environment, can we recover the 3D structure of the environment, as well as the position of the camera/robot? This has many uses in robotics and autonomous systems, as understanding the 3D structure of the environment is crucial to navigation. You don't want your robot constantly bumping into walls, or running over human beings!

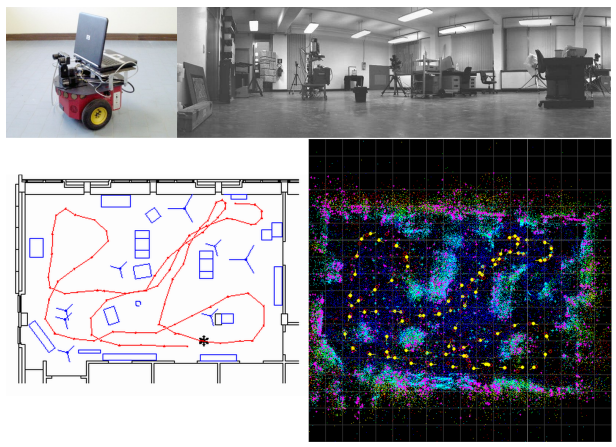

Figure 1: Example of a robot using SLAM, a 3D reconstruction and localization algorithm

In this assignment there are two programming parts: sparse reconstruction and dense reconstruction. Sparse reconstructions generally contain a number of points, but still manage to describe the objects in question. Dense reconstructions are detailed and fine grained. In fields like 3D modelling and graphics, extremely accurate dense reconstructions are invaluable when generating 3D models of real world objects and scenes.

In **Part 1**, you will be writing a set of functions to generate a sparse point cloud for some test images we have provided to you. The test images are 2 renderings of a temple from two different angles. We have also provided you with a `npz` file containing good point correspondences between the two images. You will first write a function that computes the fundamental matrix between the two images. Then write a function that uses the epipolar constraint to find more point matches between the two images. Finally, you will write a function that will triangulate the 3D points for each pair of 2D point correspondences.

Figure 2: The two temple images we have provided to you

We have provided you with a few helpful `npz` files. The file `data/some_corresp.npz` contains good point correspondences. You will use this to compute the Fundamental matrix. The file `data/intrinsics.npz` contains intrinsic camera matrices, which you will need to compute the full camera projection matrices. Finally, the file `data/temple_coords.npz` contains some points on the first image that should be easy to localize in the second image.

In **Part 2**, we utilize the extrinsic parameters computed by Part 1 to further achieve dense 3D reconstruction of this temple. You will need to compute the rectification parameters. We have provided you with `test_rectify.py` (and some helper functions) that will use your rectification function to warp the stereo pair. You will then *optionally* use the warped pair to compute a disparity map and finally a dense depth map.

In both cases, multiple images are required, because without two images with a large portion overlapping the problem is mathematically underspecified. It is for this same reason biologists suppose that humans, and other predatory animals such as eagles and dogs, have two front facing eyes. Hunters need to be able to discern depth when chasing their prey. On the other hand herbivores, such as deer and squirrels, have their eyes position on the sides of their head, sacrificing most of their depth perception for a larger field of view. The whole problem of 3D reconstruction is inspired by the fact that humans and many other animals rely on dome degree of depth perception when navigating and interacting with their environment. Giving autonomous systems this information is very useful.

2 Sparse Reconstruction

In this section, you will be writing a set of function to compute the sparse reconstruction from two sample images of a temple. You will first estimate the Fundamental matrix, compute point correspondences, then plot the results in 3D.

It may be helpful to read through Section 2.5 right now. In Section 2.5 we ask you to write a testing script that will run your whole pipeline. It will be easier to start that now

and add to it as you complete each of the questions one after the other.

2.1 Implement the eight point algorithm (10 points)

In this question, you're going to use the eight point algorithm which is covered in class to estimate the fundamental matrix. Please use the point correspondences provided in `data/some_corresp.npz`; you can load and view the contents of a `.npz` file as follows:

```
data = np.load("../data/some_corresp.npz")
print(data.files)
```

Write a function with the following signature:

```
F = eight_point(pts1, pts2, M)
```

where `pts1` and `pts2` are $N \times 2$ matrices corresponding to the (x,y) coordinates of the N points in the first and second image respectively, and `M` is a scale parameter.

- Normalize points and un-normalize F: You should scale the data by dividing each coordinate by M (the maximum of the image's width and height) using a transformation matrix T. After computing $\mathbf{F}$, you will have to "unscale" the fundamental matrix. If $x_{norm} = Tx$, then $F_{unnorm} = T^T F T$. (Note: This formula should not be confused with how homography normalization was done in assignment 2.)

- You must enforce the rank 2 constraint on $\mathbf{F}$ before unscaling. Recall that a valid fundamental matrix $\mathbf{F}$ will have all epipolar lines intersect at a certain point, meaning that there exists a non-trivial null space for $\mathbf{F}$. In general, with real points, the eight-point solution for $\mathbf{F}$ will not come with this condition. To enforce the rank 2 constraint, decompose $\mathbf{F}$ with SVD to get the three matrices $\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}$ such that $\mathbf{F} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. Then force the matrix to be rank 2 by setting the smallest singular value in $\mathbf{\Sigma}$ to zero, giving you a new $\mathbf{\Sigma}'$. Now compute the proper fundamental matrix with $\mathbf{F}' = \mathbf{U}\mathbf{\Sigma}'\mathbf{V}^T$.

- You may find it helpful to refine the solution by using local minimization. This probably won't fix a completely broken solution, but may make a good solution better by locally minimizing a geometric cost function. For this we have provided a helper function `refineF` in `helper.py` taking in $\mathbf{F}$ and the two sets of points, which you can call from `eight_point` before unscaling $\mathbf{F}$.

- Remember that the x-coordinate of a point in the image is its column entry and y-coordinate is the row entry. Also note that eight-point is just a figurative name, it just means that you need at least 8 points; your algorithm should use an over-determined system ($N > 8$ points).

- To visualize the correctness of your estimated $\mathbf{F}$, use the function `displayEpipolarF` in `python/helper.py`, which takes in $\mathbf{F}$, and the two images. This GUI lets you select a point in one of the images and visualize the corresponding epipolar line in the other image (Figure 3).

In your write-up: Please include your recovered **F** and the visualization of some epipolar lines (similar to Figure 3).

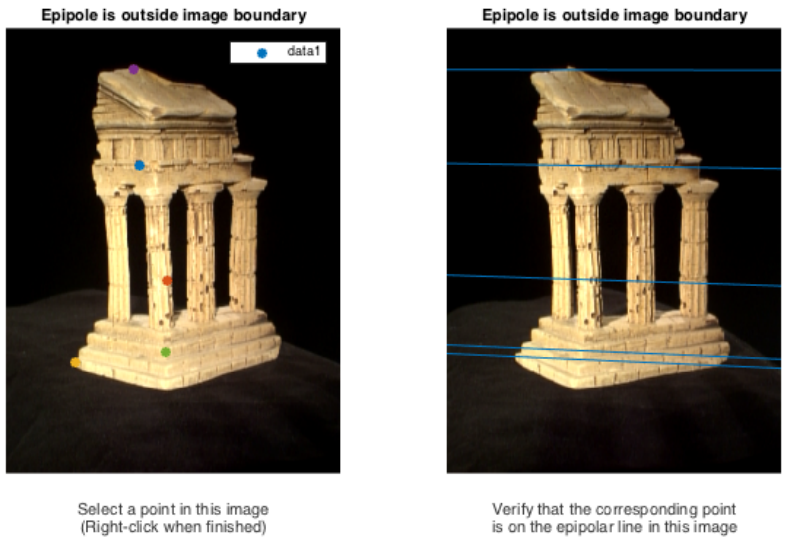

Figure 3: Epipolar lines visualization from `displayEpipolarF`

2.2 Find epipolar correspondences (20 points)

To reconstruct a 3D scene with a pair of stereo images, we need to find many point pairs. A point pair is two points in each image that correspond to the same 3D scene point. With enough of these pairs, when we plot the resulting 3D points, we will have a rough outline of the 3D object. You found point pairs in the previous homework using feature detectors and feature descriptors, and testing a point in one image with every single point in the other image. But here we can use the fundamental matrix to greatly simplify this search.

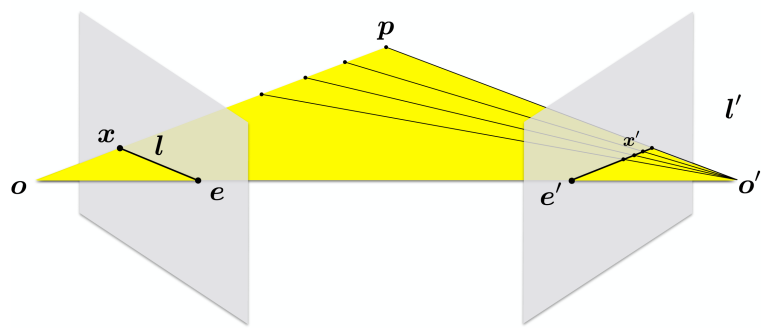

Figure 4: Epipolar Geometry (source Wikipedia)

Recall from class that given a point x in one image (the left view in Figure 4). Its corresponding 3D scene point p could lie anywhere along the line from the camera center o

to the point x. This line, along with a second image's camera center o' (the right view in Figure 4) forms a plane. This plane intersects with the image plane of the second camera, resulting in a line l' in the second image which describes all the possible locations that x may be found in the second image. Line l' is the epipolar line, and we only need to search along this line to find a match for point x found in the first image. Write a function with the following signature:

```
pts2 = epipolar_correspondences(im1, im2, F, pts1)
```

where `im1` and `im2` are the two images in the stereo pair, `F` is the fundamental matrix computed for the two images using your `eight_point` function, `pts1` is a $N \times 2$ matrix containing the (x, y) points in the first image, and the function should return `pts2`, a $N \times 2$ matrix, which contains the corresponding points in the second image.

- To match one point x in image 1, use fundamental matrix to estimate the corresponding epipolar line l' and generate a set of candidate points in the second image.
- For each candidate points x', a similarity score between x and x' is computed. The point among candidates with highest score is treated as epipolar correspondence.
- There are many ways to define the similarity between two points. Feel free to use whatever you want and **describe it in your write-up**. One possible solution is to select a small window of size w around the point x. Then compare this target window to the window of the candidate point in the second image. For the images we gave you, simple Euclidean distance or Manhattan distance should suffice.
- Remember to take care of data type and index range.

You can use the function `epipolarMatchGUI` in `python/helper.py` to visually test your function. Your function does not need to be perfect, but it should get most easy points correct, like corners, dots etc.

In your write-up: Please include a screenshot of `epipolarMatchGUI` running with your implementation of `epipolar_correspondences` (similar to Figure 5). Mention the similarity metric you decided to use. Also comment on any cases where your matching algorithm consistently fails, and why you might think this is.

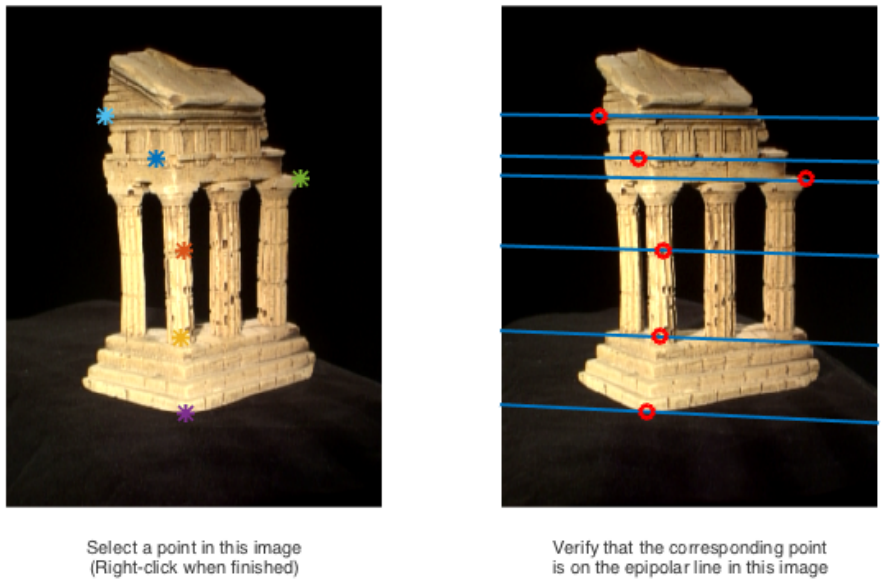

Figure 5: Epipolar Match visualization. A few errors are alright, but it should get most easy points correct (corners, dots, etc.)

2.3 Write a function to compute the essential matrix (10 points)

In order to get the full camera projection matrices we need to compute the Essential matrix. So far, we have only been using the Fundamental matrix. Write a function with the following signature:

```
E = essential_matrix(F, K1, K2)
```

Where `F` is the Fundamental matrix computed between two images, `K1` and `K2` are the intrinsic camera matrices for the first and second image respectively (contained in `data/intrinsics.npz`), and `E` is the computed essential matrix. The intrinsic camera parameters are typically acquired through camera calibration. Refer to the class slides for the relationship between the Fundamental matrix and the Essential matrix.

In your write-up: Please include your estimated `E` matrix for the temple image pair.

2.4 Implement triangulation (20 points)

Write a function to triangulate pairs of 2D points in the images to a set of 3D points with the signature:

```
pts3d = triangulate(P1, pts1, P2, pts2)
```

Where `pts1` and `pts2` are the $N \times 2$ matrices with the 2D image coordinates, `P1` and `P2` are the 3×4 camera projection matrices and `pts3d` is an $N \times 3$ matrix with the corresponding 3D points (in all cases, one point per row). Remember that you will need to multiply the given intrinsic matrices with your solution for the extrinsic camera matrices to obtain the final camera projection matrices. For `P1` you can assume no rotation or translation, so the extrinsic matrix is just $[\,\mathbf{I}\,|\,\mathbf{0}\,]$. For `P2`, pass the essential matrix to the provided function `camera2` in `python/helper.py` to get four possible extrinsic matrices. You will need to determine which of these is the correct one to use (see hint in Section 2.5). Refer to the class slides for one possible triangulation algorithm. Once implemented, check the performance by looking at the re-projection error. To compute the re-projection error, project the estimated 3D points back to the image 1 and compute the mean Euclidean error between projected 2D points and the given `pts1`.

In your write-up: Describe how you determined which extrinsic matrix is correct. Note that simply rewording the hint is not enough. Report your re-projection error using the given `pts1` and `pts2` in `data/some_corresp.npz`. If implemented correctly, the re-projection error should be less than 2 pixels.

2.5 Write a test script that uses `data/temple_coords.npz` (10 points)

You now have all the pieces you need to generate a full 3D reconstruction. Write a test script `python/test_temple_coords.py` that does the following:

1. Load the two images and the point correspondences from `data/some_corresp.npz`
2. Run `eight_point` to compute the fundamental matrix `F`
3. Load the points in image 1 contained in `data/temple_coords.npz` and run your `epipolar_correspondences` on them to get the corresponding points in image 2
4. Load `data/intrinsics.npz` and compute the essential matrix `E`.
5. Compute the first camera projection matrix $\mathtt{P}_1$ and use `camera2` to compute the four candidates for $\mathtt{P}_2$
6. Run your `triangulate` function using the four sets of camera matrix candidates, the points from `data/temple_coords.npz` and their computed correspondences.
7. Figure out the correct $\mathtt{P}_2$ and the corresponding 3D points. Hint: You'll get 4 projection matrix candidates for camera2 from the essential matrix. The correct configuration is the one for which most of the 3D points are in front of both cameras (positive depth).
8. Use matplotlib's `scatter` function to plot these point correspondences on screen.